UNIOESTE - UNIVERSIDADE ESTADUAL DO OESTE DO PARANÁ
CENTRO DE ENGENHARIAS E CIÊNCIAS EXATAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Desenvolvimento de uma Solução de Alta Disponibilidade: Uma Abordagem Prática

Guilherme Kenzo Takano Miller

FOZ DO IGUAÇU
2026

Guilherme Kenzo Takano Miller

Desenvolvimento de uma Solução de Alta Disponibilidade: Uma Abordagem Prática

Monografia submetida à Universidade Estadual do Oeste do Paraná, Curso de Ciência da Computação - Campus de Foz do Iguaçu, como requisito parcial para obtenção do grau de Bacharel em Ciência da Computação.

Ma. Teresinha Arnauts Hachisuca

Me. Filipe Ventura Muggiati

FOZ DO IGUAÇU

2026

Guilherme Kenzo Takano Miller

Desenvolvimento de uma Solução de Alta Disponibilidade: Uma Abordagem Prática

---

Ma. Teresinha Arnauts Hachisuca
Orientador(a)

Me. Filipe Ventura Muggiati
Coorientador(a)

Titulação Nome 1º membro da banca
Membro

Titulação Nome 2º membro da banca
Membro

Dedico este trabalho....

Agradecimentos

Deus, Pais, orientador, instituição, colegas... A todos que de alguma forma, contribuiram no processo

“xxx.”
(Autor)

Resumo

Escreva aqui um breve resumo do projeto de TCC Introdução sobre o assunto, proposta e resultados.

Palavras-chave:

Lista de ilustrações

Figura 1 – Exemplo de inclusão de figura 2
Figura 2 – Exemplos de figuras lado a lado. a) corresponde a um exemplo de inclusão de figura a esquerda e b) um exemplo a direita. 2
Figura 3 – Exemplo figura esquerda 3
Figura 4 – Exemplo figura direita 3

Figura 5 – Cluster de Servidores Antes do Particionamento 20
Figura 6 – Cluster de Servidores Durante um Particionamento 20
Figura 7 – Diagrama da Arquitetura do Sistema 22

Lista de tabelas

Tabela 1 – Exemplo de tabela 1 3

Tabela 2 – Comparativo entre os algoritmos Paxos e Raft 14

Tabela 3 – Métricas da Falha no Banco de Dados 26

Tabela 4 – Métricas do Particionamento de Rede 27

Lista de Abreviaturas e Siglas

A_i Área do i^{th} elemento

ANN *Artificial Neural Network*

CFT Crash Fault Tolerant

DCS Distributed Configuration Store

HA High Availability

HMI Human Machine Interface

HTTP/2 Hypertext Transfer Protocol 2

JDBC Java Database Connectivity

JSON JavaScript Object Notation

JVM Java Virtual Machine

MTBF Mean Time Between Failures

MTTF Mean Time to Failure

MTTR Mean Time to Repair

PLCs Programmable Logic Controllers

REST Representational State Transfer

RPC Remote Procedure Call

RTUs Remote Terminal Units

SCADA Supervisory Control and Data Acquisition

SQL Structured Query Language

TCC Trabalho de Conclusão de Curso

VM Virtual Machine

WAL Write-Ahead Logging

WEKA *Waikato Environment for Knowledge Analysis*

Sumário

1 Introdução 1

2 Exemplos selecionados 2

2.1 Considerações Iniciais 2
2.2 Uso das ”Aspas”/ “Aspas” 2
2.3 Figuras 2
2.4 Tabelas 3
2.5 Equações 3
2.5.1 Equações simples 3
2.5.2 Equações mais robustas 4
2.6 Matrizes 4
2.7 Notas de rodapé 4
2.8 Nomenclaturas / Siglas 4
2.9 Referências Cruzadas 5
2.10 Referências Bibliográficas 5
2.11 Considerações Finais 6

3 Revisão Bibliográfica 7

3.1 Sistemas SCADA e a Exigência de Disponibilidade 7
3.1.1 Arquitetura e Criticidade 7
3.1.2 Desafios de HA em Ambientes SCADA 8
3.2 Alta Disponibilidade 8
3.2.1 Definição e Importância 8
3.2.2 Métricas de Disponibilidade 9
3.2.3 Pilares da Alta Disponibilidade 9
3.3 Consenso em Sistemas Distribuídos 10
3.3.1 Problemas de Consenso 10
3.3.2 Algoritmo de Paxos 11
3.3.3 Algoritmo de Raft 13
3.3.4 Análise Comparativa 13
3.4 Alta Disponibilidade em Bancos de Dados PostgreSQL 14
3.4.1 PostgreSQL 14
3.4.2 Replicação de Dados 15
3.4.3 Consistência de Dados em Ambientes Replicados 15
3.4.4 Ferramentas de Código Aberto para Automação de *Failover* 16

3.5 Protocolos de Comunicação 17
3.6 Considerações do Capítulo 18

4 Metodologia e Arquitetura Proposta 19
4.1 Visão Geral da Solução 19
4.2 Caracterização e Detecção de Falhas 19
4.2.1 Tipos de Falhas 19
4.2.2 Mecanismos de Detecção 21
4.3 Caracterização do Ambiente SCADA 21
4.4 Arquitetura de Banco de Dados e Consenso 21
4.5 Dinâmica de Failover 23
4.5.1 Prevenção de Split-Brain 23
4.5.2 Redirecionamento de Clientes 23
4.6 Ambiente de Simulação 24
4.6.1 Limitações da Simulação 25

5 Testes Realizados 26
5.1 Análise do Cenário 1: Falha no Banco de Dados 26
5.2 Análise do Cenário 2: Particionamento de Rede 27
5.3 Limitações da Arquitetura 27
5.3.1 Modelo Crash Fault Tolerance 27
5.3.2 Problema de Reconexão 28
5.4 Considerações Finais do Capítulo 28

6 Conclusões e Trabalhos Futuros 29

Referências 30

1 Introdução

Esse capítulo deveria ser composto por: introdução, estado da arte, proposta, justificativa, objetivos e organização da monografia. Uma introdução, como o próprio nome já diz, tem a função de introduzir o leitor ao tema, isto é, a partir dela tem-se uma visão total do trabalho de forma sucinta e objetiva.

É mais comum escrever este capítulo após a finalização de toda a pesquisa. Entretanto se esse capítulo for escrito juntamente com o início das pesquisas, é provável que sejam necessárias alterações no decorrer do desenvolvimento do Trabalho de Conclusão de Curso (`TCC`) .

Observações/Dicas:

1. expor as ideias de forma clara, concisa e convidativa: o leitor precisa se interessar pelo tema;
2. manter a coerência: redija um texto com início, meio e fim;
3. referenciar todo o conteúdo pesquisado;
4. evitar escrever em primeira pessoa (singular/plural);
5. ler o texto em voz alta, isso normalmente ajuda a identificar problemas de semântica e normalmente facilita a organização do conteúdo;

2 Exemplos selecionados

2.1 Considerações Iniciais

Nesse capítulo são apresentados alguns exemplos de comandos em LaTeX que objetivam facilitar o processo de escrita da monografia.

2.2 Uso das "Aspas"/ “Aspas”

Para que as aspas fiquem certas no LaTeX precisa-se abri-las com duas crases seguidas. Pode-se ver no título dessa seção o uso incorreto contrastando com o correto.

2.3 Figuras

Para centralizar a inclusão de figuras, foram criados dois comandos em LaTeX que já incoporam o padrão brasileiro de inclusão de imagens: `\conteudoFigura` e `\duasFiguras`.

Os formatos de figuras testados para esse comando foram `.png` ou `.pdf`.

Na Figura 1 é exibido um exemplo de inclusão simples de figura.



Figura 1 – Exemplo de inclusão de figura

Fonte: Unioeste [2017]

Os exemplo exibido através da 2 ilustra como dispor figuras lado a lado caso o autor das imagens seja o mesmo.



a) Figura esquerda



b) Figura direita

Figura 2 – Exemplos de figuras lado a lado. a) corresponde a um exemplo de inclusão de figura a esquerda e b) um exemplo a direita.

Fonte: Unioeste [2017]

Os exemplos exibidos através das Figuras 3 e 4 ilustram como dispor figuras lado a lado. Esse exemplo pode ser utilizado quando as fontes das figuras correspondem a autores diferentes.



Figura 3 – Exemplo figura esquerda

Fonte: Unioeste [2017]



Figura 4 – Exemplo figura direita

Fonte: Unioeste [2017]

É possível organizar as imagens de outras formas, rotacioná-las, elaborá-las a partir do próprio LaTeX, tudo depende da necessidade de cada um.

2.4 Tabelas

A Tabela 1 é um exemplo de tabela construída em LaTeX.

Tabela 1 – Exemplo de Tabela 1

Coluna 1	**Coluna 2**	**Coluna 3**	**Coluna ....**
Texto 1 1	Texto 1 2	Texto 1 3	Texto 1 4
Texto 2 1	Texto 2 2	Texto 2 3	Texto 2 4
Texto 3 1	Texto 3 2	Texto 3 3	Texto 3 4

Fonte: Autor

2.5 Equações

Nessa seção são dispostos alguns exemplos de equações. Elas servem somente como base. Caso seja necessária a inclusão de equações mais complexas na monografia, deverá ser utilizar outro material como base.

2.5.1 Equações simples

Quando deseja-se incluir um exemplo de equação na mesma linha pode utilizar os comandos `\(equação\)`. Por exemplo o teorema de Pitágoras $a^2 + b^2 = c^2$. Caso deseje-se que a mesma seja exibida na linha seguinte deve-se mudar os parênteses por chaves `\[equação\]`:

$$a^n + b^n = c^n$$

Outra forma simples de apresentar uma equação é através do uso do dólar para iniciar e fechar o modo de edição de equação: `$E=mc^2$`, resultando em $E = mc^2$.

Nesse tipo de exemplo não é gerada referência para a equação.

2.5.2 Equações mais robustas

Por exemplo a Equação 2.1, que é a fórmula de Einstein:

$$e = m \cdot c^2 \ , \tag{2.1}$$

que é ao mesmo tempo a mais conhecida e a menos compreendida fórmula física.

De onde segue a lei actual de Kirchhoff:

$$\sum_{k=1}^{n} I_k = 0 \ . \tag{2.2}$$

2.6 Matrizes

Estão listados aqui três exemplo sismples de criação de matriz.

$$A = \begin{bmatrix} 1 & 3 & 0 \\ 2 & 4 & -2 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 3 & -2 \end{bmatrix}, \quad \text{e} \quad C = \begin{bmatrix} 1 \\ 4 \\ -3 \end{bmatrix}$$

2.7 Notas de rodapé

As notas de rodapé são anotações extras vinculadas a um determinado texto. A nota de rodapé pode ser inserida após a palavra ou frase a qual se refere através do comando: `\footnote`{texto}.

Exemplo: Nota de radapé[1].

2.8 Nomenclaturas / Siglas

Ao usar pela primeira vez uma sigla não se deve esquecer de fornecer sua descrição e criar a entrada na lista de siglas. Um exemplo da sigla `TCC` pode ser observado no Capítulo Introdução, página 1 deste documento.

Não há problemas em repetir as siglas, o próprio LaTeX de encarrega de remover a duplicidade. O LaTeX também as organiza de forma ordenada.

Exemplos de fórmulas e texto em itálico: Área do i^{th} elemento (A_i); *Artificial Neural Network* (`ANN`) .

[1] nota de rodapé

2.9 Referências Cruzadas

Pode-se usar o comando `\label`{identificador} para marcar diferentes locais no texto, tais como capítulos, seções, figuras, tabelas, equações... Para referenciar um local identificado deve-se usar o comandos `\ref` {identificador} ou `\pageref` {identificador}. No inicio dessa seção foi criado uma referência a mesma e a seguir um exemplo do uso dos dois comandos para usar a referência.

Pode-se ver um exemplo de referência na Seção 2.9 da página 5.

É possível também criar um *link* para a página referenciada através do comando `\hyperref`. A sintaxe é: `\hyperref`[identificador]{texto}.

Se o identificador já foi definido então será criado um *link* para o mesmo, caso contrário será apresentado somente o texto.

Exemplo de link: Link para o capítulo Introdução.

2.10 Referências Bibliográficas

Ha diferenças em se realizar citações simples no final da frase, múltiplas citações ou citação no meio da frase/parágrafo.

Abaixo estão relacionados alguns exemplos das formas mais utilizadas:

- Citação simples no final da frase: coloque o texto, não pode ser a cópia exata, mas sim a idéia que representa o texto. O *Waikato Environment for Knowledge Analysis* (`WEKA`) é uma ferramenta de código aberto que pode ser incorporado nas mais diversas aplicações [HALL et al., 2009];
- Citação literal: "O `WEKA` é uma ferramenta de código aberto que pode ser incorporado nas mais diversas aplicações" [HALL et al., 2009]. Há outras formas de se fazer citação literal, converse com seu orientador caso opte por outro formato;
- Citações múltiplas: basta acrescentar vírgula entre as citações dentro do comando *cite*. Exemplo: [HALL et al., 2009; JONES et al., 2001];
- Usar o nome do autor como terceira pessoa: Conforme Jones et al. [2001], *open source* é ...

No arquivo `Geral/bibliografia.bib` há diversos tipos de referências, exemplificados conforme o tipo de documento referenciado.

2.11 Considerações Finais

Esse Capítulo teve por objetivo apresentar uma miscelânea de comandos LaTeX mais utilizados na elaboração da monografia de `TCC` do curso de Ciência da Computação da Unioeste, Campos de Foz do Iguaçu.

Espera-se que o mesmo tenha contribuído positivamente para o desenvolvimento de sua monografia.

3 Revisão Bibliográfica

Neste capítulo, serão apresentados primeiramente os conceitos referentes aos sistemas *Supervisory Control and Data Acquisition* (SCADA), que contextualizam a aplicação prática e a relevância deste trabalho. Em seguida, serão detalhados os fundamentos teóricos e práticos sobre Alta Disponibilidade (HA - High Availability), consenso em sistemas distribuídos, bancos de dados PostgreSQL e protocolos de comunicação, pilares essenciais para a arquitetura que será desenvolvida ao longo do projeto.

3.1 Sistemas SCADA e a Exigência de Disponibilidade

Sistemas SCADA são a base de infraestruturas críticas modernas, como redes de energia elétrica, estações de tratamento de água e processos industriais complexos [ALANAZI; MAHMOOD; CHOWDHURY, 2023]. Por sua natureza, esses sistemas não apenas processam dados, mas interagem diretamente com o mundo físico através do controle de atuadores. Consequentemente, a exigência de disponibilidade contínua não é apenas um requisito de desempenho, mas uma necessidade fundamental para garantir a segurança operacional, a eficiência e a proteção contra falhas catastróficas.

3.1.1 Arquitetura e Criticidade

Um sistema SCADA é projetado para monitorar e controlar processos industriais que podem estar geograficamente distribuídos. Sua arquitetura é tipicamente organizada em camadas, começando pelos dispositivos de campo, como sensores e atuadores, que interagem diretamente com o processo físico. As informações coletadas por esses sensores são enviadas para Unidades Terminais Remotas (RTUs) ou Controladores Lógicos Programáveis (PLCs), que executam o controle local [YADAV; PAUL, 2021].

Esses dados são então transmitidos através de uma rede de comunicação para uma estação central de supervisão, onde uma Interface Homem-Máquina (HMI) permite que operadores humanos monitorem o estado de todo o processo e intervenham quando necessário [YADAV; PAUL, 2021]. Os bancos de dados desempenham um papel central nesta arquitetura, armazenando tanto os dados históricos do processo para análise quanto os dados de configuração em tempo real que são vitais para a operação.

A criticidade dos sistemas SCADA é imensa. Eles são classificados como parte da infraestrutura crítica nacional em muitos países, pois uma falha ou um comprometimento pode resultar em interrupções no fornecimento de energia, contaminação da água potável, acidentes industriais e perdas financeiras significativas [ALANAZI; MAHMOOD;

CHOWDHURY, 2023]. Qualquer período de indisponibilidade, mesmo que breve, pode impedir que os operadores visualizem o estado do sistema ou controlem os equipamentos, levando a condições operacionais inseguras.

3.1.2 Desafios de HA em Ambientes SCADA

A necessidade de operação ininterrupta impõe desafios rigorosos de alta disponibilidade (`HA`) aos sistemas `SCADA`. A tolerância a falhas deve ser implementada em todas as camadas da arquitetura para mitigar pontos únicos de falha. Isso inclui a redundância de sensores, redes de comunicação, controladores e, de forma crucial, dos servidores e bancos de dados que compõem o sistema de supervisão.

Um dos principais desafios é garantir um mecanismo de *failover* rápido e confiável. Quando um servidor primário ou um banco de dados falha, um sistema secundário deve assumir suas funções de forma transparente e quase instantânea para não comprometer o controle do processo [YADAV; PAUL, 2021]. Em ambientes críticos que possuem mais de uma localização (*multi-site*) para o armazenamento da sua infraestrutura, a complexidade aumenta com a necessidade de redundância geográfica, que protege contra falhas que afetam um local físico inteiro, como incêndios ou desastres naturais.

Além das falhas de *hardware* e *software*, os sistemas `SCADA` modernos enfrentam ameaças de segurança cibernética que podem comprometer a disponibilidade [ALANAZI; MAHMOOD; CHOWDHURY, 2023]. Um ataque bem-sucedido pode não apenas roubar dados, mas também paralisar as operações, tornando a integração de estratégias de segurança e alta disponibilidade um desafio complexo. Portanto, a implementação de uma solução de `HA` deve considerar não apenas a detecção de falhas de componentes, mas também a resiliência contra eventos imprevistos, garantindo a consistência e a disponibilidade contínua dos dados que alimentam as decisões de controle.

3.2 Alta Disponibilidade

3.2.1 Definição e Importância

Conforme Rajesh e Kumar [2025], a alta disponibilidade é a capacidade de um sistema permanecer operacional pelo maior tempo possível. Em sistemas distribuídos, esse requisito de projeto torna-se ainda mais crítico, pois envolve estratégias de redundância, tolerância a falhas, mecanismos de migração de servidores (*failover*), replicação e consistência de dados, monitoramento, balanceamento de carga e recuperação para minimizar janelas de indisponibilidade (*downtime*).

Com isso, é possível afirmar que para desenvolver um sistema altamente disponível, é necessário combinar um conjunto de tecnologias, estratégias e conceitos, aplicados para

garantir a continuidade de um sistema.

No atual cenário tecnológico, a continuidade operacional dos sistemas é um requisito não funcional crítico, dado que a falha desses sistemas, na maioria dos cenários, acarreta diversos problemas, como por exemplo: perda de informações em tempo real e atrasos na comunicação entre clientes e servidores.

3.2.2 Métricas de Disponibilidade

O artigo de Toeroe e Tam [2012] mostra três conceitos utilizados em conjunto para a medição da disponibilidade de um sistema e por consequência, auxiliam a medir a eficiência das estratégias e tecnologias utilizadas para garantir o maior tempo de disponibilidade de um sistema.

- *Mean Time Between Failures* (`MTBF`): a sigla em inglês corresponde ao tempo médio entre falhas, é a média dos intervalos de tempo entre falhas consecutivas em um sistema reparável. Toeroe e Tam [2012] ressaltam que essa métrica só é válida e aplicável para sistemas que podem ser consertados, pois um sistema só pode falhar uma segunda vez se tiver sido devidamente reparado após o primeiro incidente;
- *Mean Time to Repair* (`MTTR`): é conceituado como o tempo esperado e necessário para consertar o sistema para que ele retorne à sua operação correta e normal. Toeroe e Tam [2012] também explica que é difícil medir com precisão o tempo para reparar o sistema, porém uma das formas de contornar essa dificuldade seria causar falhas propositais no sistema e depois realizar o cálculo com base no tempo para repará-lo;
- *Mean Time to Failure* (`MTTF`): tempo esperado para que o sistema venha a falhar. Essa métrica equivale ao tempo de atividade da aplicação ou *uptime*, correspondendo ao período em que o sistema está operacional e fornecendo o seu serviço aos usuários.

Gray e Siewiorek [1991] também mostra um conceito que é utilizado para definir se um sistema é dado como altamente disponível. O termo em questão é chamado de os cinco noves ou 99,999%, esse termo faz referência ao tempo de *uptime* do sistema, os cinco noves permitem aproximadamente no máximo cinco minutos de *downtime* por ano.

3.2.3 Pilares da Alta Disponibilidade

De acordo com Rajesh e Kumar [2025] existem alguns pilares que devem ser levados em conta ao planejar um sistema altamente disponível, dentre eles os mais relevantes para esse trabalho são:

1. **Tolerância a Falhas**: é a base da alta disponibilidade, permite que os sistemas continuem funcionando apesar de falhas em componentes individuais. Para isso, são incorporados componentes redundantes, sistemas de *backup* e protocolos de tratamento de erros, garantindo que as falhas não interrompam a disponibilidade do serviço;

2. **Redundância**: envolve a duplicação de componentes ou serviços críticos para eliminar pontos únicos de falha. Isso pode incluir *clusters*[1] de servidores, replicação de dados e fontes de alimentação de *backup*. Os sistemas redundantes operam em paralelo, garantindo que um nó redundante assuma as operações sem interrupção em caso de falha do nó primário;

3. **Mecanismos de *Failover***: se refere ao processo de transferir cargas de trabalho automaticamente de um componente que falhou para um componente de *backup*;

4. **Replicação e Consistência**: devido à redundância, torna-se necessária a duplicação das bases de dados, com isso é fundamental assegurar que os dados sejam consistentes em ambas as bases utilizadas;

3.3 Consenso em Sistemas Distribuídos

No âmbito de sistemas distribuídos, o problema do consenso consiste em fazer com que um conjunto de processos, que podem sofrer falhas, chegue a um acordo sobre um único valor [LAMPORT, 2001].

A importância do consenso reside na sua capacidade de resolver conflitos, alguns exemplos de problemas que ele consegue resolver são:

- Réplicas de bancos de dados devem concordar em uma sequência de requisições para manter os dados consistentes [SOARES, 2010];

- Eleger um líder em um *cluster*. Nessa situação é necessário que a maioria dos nós concordem com a eleição de um líder [BHARDWAJ; DATTA, 2020].

3.3.1 Problemas de Consenso

Atingir o consenso em um sistema distribuído é uma tarefa desafiadora. Os algoritmos devem ser resilientes a uma variedade de falhas, como falhas de nós, conhecidas como *crash*, particionamento da rede, atraso, perda ou duplicação de mensagens [BHARDWAJ; DATTA, 2020].

[1] Conjunto de computadores interconectados que trabalham em conjunto, operando como um único sistema

Dentre os problemas propostos no estudo do consenso em sistemas distribuídos, um dos mais conhecidos é o Problema dos Generais Bizantinos. Proposto por Lamport, Shostak e Pease [1982], consiste em um cenário onde alguns componentes do sistema podem falhar de maneira arbitrária ou maliciosa, enviando assim informações conflitantes para diferentes partes do sistema. Um algoritmo de consenso robusto precisa assegurar que os nós honestos cheguem a um acordo, mesmo na presença desses nós maliciosos.

Outro grande problema nessa área ocorre devido a uma falha citada anteriormente: o particionamento de rede. Essa falha ocorre por algum problema na infraestrutura de comunicação, possuindo uma relação de causa e efeito severo dentro do campo de sistemas distribuídos. Um dos efeitos causados por essa falha é o *split-brain*. Segundo Kleppmann [2017], esse fenômeno faz com que dois nós fiquem isolados um do outro e acreditem, simultaneamente, que são o líder do sistema. Se ambos os nós continuarem aceitando operações de gravação de forma independente, o banco de dados sofrerá corrupção permanente e divergência de informações, pois não haverá uma forma segura de mesclar esses dados conflitantes posteriormente. É exatamente para prevenir esse tipo de catástrofe que algoritmos como o Raft exigem um quórum, garantindo que um líder só possa ser eleito se receber a maioria absoluta dos votos.

3.3.2 Algoritmo de Paxos

O algoritmo Paxos [LAMPORT, 2001], proposto por Leslie Lamport, é um dos mais influentes para alcançar consenso em sistemas distribuídos tolerantes a falhas. Embora tenha a reputação de ser complexo, sua lógica fundamental deriva de maneira direta das propriedades de segurança que um sistema de consenso precisa satisfazer. O algoritmo foi projetado para operar em um ambiente assíncrono, onde os processos podem falhar por interrupção, também chamadas de *crash failure*, reiniciando, e as mensagens podem sofrer atrasos, perdas ou duplicações, mas não são corrompidas.

Para simplificar a explicação, o algoritmo divide os processos em três papéis distintos:

- ***Proposers***: São os agentes que propõem um valor a ser acordado;
- ***Acceptors***: São os agentes que podem aceitar os valores propostos. Um valor é escolhido quando uma maioria de *Acceptors* o aceita. Eles são a memória do protocolo e devem manter o estado em armazenamento estável para sobreviver a falhas;
- ***Learners***: São os agentes que aprendem qual valor foi efetivamente escolhido pelo consenso.

Em uma implementação prática, um único processo pode desempenhar todos os três papéis. O objetivo é assegurar que apenas um único valor seja escolhido e que, uma vez escolhido, os processos possam aprender qual foi esse valor. Para isso, o Paxos se baseia em um protocolo de duas fases, sendo elas a preparação e a aceitação.

A primeira fase é a preparação (*prepare*), ela consiste em duas etapas principais:

- Um *Proposer* seleciona um número de proposta n, que deve ser único e maior que qualquer número de proposta anterior usado por ele. Em seguida, envia uma mensagem de preparação, *prepare*(n), para uma maioria de *Acceptors*;
- Um *Acceptor* que recebe a mensagem *prepare*(n) verifica se n é maior do que qualquer número de proposta de uma mensagem de preparação que ele já tenha respondido. Se for, o *Acceptor* promete não aceitar mais nenhuma proposta com número menor que n e responde ao *Proposer* com a proposta de maior número (se houver) que ele já aceitou.

A segunda fase consiste na aceitação (*accept*), que também pode ser dividida em duas etapas:

- Se o *Proposer* receber respostas de uma maioria de *Acceptors*, ele está autorizado a enviar uma mensagem de aceitação, *accept*(n, v), para eles. O valor v da proposta é determinado da seguinte forma: se os *Acceptors* retornaram propostas previamente aceitas, o *Proposer* deve escolher o valor da proposta com o maior número entre as respostas. Se nenhum *Acceptor* havia aceitado uma proposta antes, o *Proposer* é livre para escolher qualquer valor que ele deseje propor;
- Um *Acceptor* que recebe a mensagem *accept*(n, v) a aceita, a menos que já tenha respondido a uma mensagem de preparação com um número maior que n.

A propriedade crucial que garante a segurança (que apenas um único valor seja escolhido) é que qualquer proposta escolhida por uma maioria de *Acceptors* em uma rodada influenciará a escolha do valor em todas as rodadas subsequentes. Como qualquer maioria em duas rodadas diferentes terá pelo menos um *Acceptor* em comum, o protocolo garante que um novo valor só pode ser proposto se ele for o mesmo que o valor da rodada anterior que já pode ter sido decidido.

Para assegurar o progresso (*liveness*), o algoritmo geralmente elege um único processo para atuar como um líder (ou *distinguished proposer*), que é o único a emitir propostas. Se o líder conseguir se comunicar com uma maioria de *Acceptors*, ele eventualmente terá sucesso em fazer com que um valor seja escolhido. Se múltiplos líderes tentarem

propor valores simultaneamente, é possível que o sistema não chegue a um consenso, mas a segurança nunca é violada.

Embora correto e influente, o Paxos é reconhecidamente difícil de implementar em sua forma original [ONGARO; OUSTERHOUT, 2014]. Essa dificuldade motivou o desenvolvimento do algoritmo Raft, apresentado a seguir.

3.3.3 Algoritmo de Raft

O Raft [ONGARO; OUSTERHOUT, 2014] foi desenvolvido com o objetivo principal de ser mais compreensível que o Paxos, sem sacrificar a eficiência ou a correção. Ele produz um resultado equivalente ao Paxos e é igualmente eficiente, mas sua estrutura é fundamentalmente diferente. Para facilitar o entendimento, o Raft decompõe o problema do consenso em três subproblemas relativamente independentes:

1. **Eleição de Líder (*Leader Election*)**: O Raft utiliza uma forma mais forte de liderança do que outros algoritmos. O sistema sempre possui um único líder, e todos os outros nós são seguidores (*Followers*). O líder é responsável por gerenciar o *log* replicado e interagir com os clientes. Se um seguidor para de receber mensagens de "*heartbeat*"do líder após um certo tempo (um *election timeout* aleatório), ele se torna um candidato e inicia uma nova eleição. Um candidato vence se receber votos da maioria dos nós;

2. **Replicação de *Log* (*Log Replication*)**: O líder é o único responsável por gerenciar o *log* replicado. Ele recebe os comandos dos clientes, os adiciona ao seu próprio *log* e os replica para os seguidores. No Raft, o fluxo de dados do *log* é estritamente unidirecional: do líder para os seguidores. O líder força os *logs* dos seguidores a espelharem o seu, sobrescrevendo quaisquer inconsistências. Um comando é considerado confirmado (*committed*) quando é replicado com sucesso na maioria dos servidores;

3. **Segurança (*Safety*)**: Para assegurar que a máquina de estados de cada nó execute os mesmos comandos na mesma ordem, o Raft adiciona uma restrição à eleição de líder: um candidato não pode vencer uma eleição a menos que seu *log* contenha todas as entradas já confirmadas no *cluster*. Isso garante que um novo líder sempre terá o estado mais completo e atualizado.

3.3.4 Análise Comparativa

Paxos e Raft são ambos algoritmos projetados para resolver o problema de consenso em sistemas tolerantes a falhas, mas eles o fazem com abordagens e focos diferentes.

Critério	**Paxos**	**Raft**
Abordagem	*Peer-to-peer* simétrica; o líder é uma otimização, não um requisito para o funcionamento do algoritmo.	Centrada no líder; todo o fluxo de consenso e replicação parte de um único nó eleito.
Compreensibilidade	Reconhecidamente difícil de implementar corretamente em sua forma original [ONGARO; OUSTERHOUT, 2014].	Projetado com o objetivo explícito de ser compreensível; decompõe o consenso em subproblemas [ONGARO; OUSTERHOUT, 2014].
Tolerância a Falhas	Tolera *crash failures*; não tolera falhas bizantinas [LAMPORT, 2001].	Tolera *crash failures*; não tolera falhas bizantinas em sua forma padrão [ONGARO; OUSTERHOUT, 2014].
Eficiência	Desempenho comparável ao Raft, apesar da maior complexidade de implementação.	Desempenho equivalente ao Paxos; estrutura mais simples facilita otimizações [ONGARO; OUSTERHOUT, 2014].
Escalabilidade	A troca de mensagens pode se tornar um gargalo com o crescimento do número de nós, exigindo otimizações complexas [AILIJIANG; CHARAPKO; DEMIRBAS, 2019].	Limitada pelo líder único; todo o fluxo de operações depende do desempenho de um único nó [ONGARO; OUSTERHOUT, 2014].

Tabela 2 – Comparativo entre os algoritmos Paxos e Raft

Fonte: Autor (2026)

3.4 Alta Disponibilidade em Bancos de Dados PostgreSQL

A implementação de alta disponibilidade em sistemas de gerenciamento de bancos de dados é essencial para garantir a continuidade das operações em aplicações críticas. No ecossistema do PostgreSQL [GROUP, 2026], a alta disponibilidade é alcançada através de um conjunto robusto de funcionalidades nativas e ferramentas de código aberto que trabalham em conjunto para prover replicação, consistência e mecanismos de *failover* automatizados.

3.4.1 PostgreSQL

O PostgreSQL é um sistema de gerenciamento de banco de dados relacional de código aberto, reconhecido por sua robustez, extensibilidade e conformidade com os

padrões *Structured Query Language* (SQL). Graças a décadas de desenvolvimento ativo por uma comunidade global, ele se tornou uma escolha popular para aplicações que exigem alta confiabilidade e integridade de dados, desde serviços web em larga escala até sistemas industriais críticos. Sua arquitetura foi projetada para ser resiliente, e seu mecanismo de *logs* de transação *Write-Ahead Logging* (WAL) serve como a base para suas estratégias avançadas de replicação e recuperação de desastres.

3.4.2 Replicação de Dados

A replicação de dados é o pilar da alta disponibilidade no PostgreSQL, permitindo que cópias de um banco de dados primário sejam mantidas em um ou mais servidores secundários. O PostgreSQL oferece principalmente dois métodos de replicação:

- **Replicação Física**: É o método mais comum e eficiente. Baseia-se na transmissão contínua dos registros do WAL do servidor primário para os servidores secundários. Como o WAL contém uma descrição de baixo nível de todas as modificações no banco de dados, os servidores secundários podem reconstruir o estado exato do primário, *byte a byte*. Essa abordagem garante uma cópia fiel e é otimizada para desempenho;
- **Replicação Lógica**: Diferente da replicação física, este método replica as alterações com base em sua representação lógica (comandos *INSERT*, *UPDATE*, *DELETE*). Essa abordagem oferece maior flexibilidade, permitindo, por exemplo, a replicação de um subconjunto de tabelas, a consolidação de múltiplos bancos de dados em um só, ou a replicação entre diferentes versões do PostgreSQL e até mesmo entre sistemas operacionais distintos.

3.4.3 Consistência de Dados em Ambientes Replicados

Em um *cluster* de replicação, é fundamental definir o quão consistentes os servidores secundários precisam estar em relação ao primário. O PostgreSQL permite configurar essa consistência através de dois modos de replicação, que representam um *trade-off* entre durabilidade e latência:

- **Replicação Assíncrona**: Neste modo, o servidor primário confirma uma transação ao cliente assim que ela é escrita em seu próprio disco, sem esperar pela confirmação dos servidores secundários. Esta é a configuração padrão, pois oferece o melhor desempenho e a menor latência. No entanto, ela introduz um pequeno risco de perda de dados: se o servidor primário falhar antes que as últimas transações sejam enviadas e aplicadas nos secundários, essas transações podem ser perdidas;

- **Replicação Síncrona**: Para eliminar o risco de perda de dados, a replicação pode ser configurada de forma síncrona. Com esta configuração, o servidor primário espera até que pelo menos um servidor secundário confirme que recebeu e escreveu as alterações em disco antes de confirmar a transação ao cliente. Isso garante que a transação sobrevive à falha do primário, mas ao custo de uma latência maior, já que a confirmação depende da comunicação pela rede com outro servidor.

3.4.4 Ferramentas de Código Aberto para Automação de *Failover*

A replicação por si só não garante a alta disponibilidade. Quando o servidor primário falha, é necessário um mecanismo que detecte a falha e promova um dos servidores secundários para assumir o papel de novo primário. Esse processo é conhecido como *failover*. Embora possa ser feito manualmente, a automação é crucial para garantir uma recuperação rápida e sem erros. A comunidade PostgreSQL desenvolveu diversas ferramentas robustas para essa finalidade:

- **Patroni**: É uma das soluções mais populares e robustas para automação de alta disponibilidade. Ele atua como um molde, utilizando um Armazenamento de Configuração Distribuída (`DCS`), como etcd, Consul ou ZooKeeper, para gerenciar o estado do *cluster* de forma consistente. O Patroni monitora a saúde dos nós e, em caso de falha do líder, orquestra uma eleição para promover o seguidor mais atualizado, automatizando o *failover* de forma segura [SE, 2026];
- **repmgr**: É uma ferramenta de código aberto focada no gerenciamento de *clusters* de replicação e *failover* do PostgreSQL. Ela fornece um conjunto de utilitários para configurar, monitorar e executar operações de *failover* e *switchover* (uma troca de papéis planejada). O **repmgr** simplifica muitas das tarefas administrativas complexas associadas à manutenção de um *cluster* replicado [2NDQUADRANT, 2026];
- **pg_auto_failover**: É uma solução que busca simplificar a configuração de um *cluster* PostgreSQL com *failover* automático. Ele introduz um nó monitor que gerencia a saúde dos servidores de dados e toma decisões de *failover*. Diferente do Patroni, ele não depende de um `DCS` externo, o que torna sua arquitetura e configuração mais simples, sendo uma alternativa adequada para ambientes que necessitam de alta disponibilidade sem uma complexidade excessiva [DATA, 2026].

No entanto, para que os mecanismos de replicação e *failover* automático funcionem corretamente, os nós do banco de dados precisam trocar informações constantemente. Essa troca de mensagens depende diretamente da infraestrutura de rede e dos protocolos de comunicação utilizados, o que evidencia a necessidade de entender como essas tecnologias operam em sistemas distribuídos.

3.5 Protocolos de Comunicação

Para analisar o impacto da infraestrutura de rede na resiliência de um sistema, torna-se necessário segmentar a comunicação em duas camadas arquiteturais distintas, sendo a primeira voltada à aquisição de dados no domínio industrial e a segunda destinada à orquestração da alta disponibilidade. Essa divisão conceitual é fundamental porque os requisitos de consistência, volume de dados e tolerância a falhas diferem drasticamente entre o monitoramento do chão de fábrica e a sincronização dos nós de banco de dados.

Na camada de automação industrial, o protocolo OPC-UA atua como o padrão para a integração vertical entre os equipamentos de campo e os sistemas de supervisão. Conforme abordado por Drahoš et al. [2018], esse protocolo provê um modelo de informações semântico que contextualiza as variáveis coletadas, além de incorporar mecanismos nativos de criptografia e autenticação. Contudo, o transporte dessa estrutura de metadados eleva o tamanho dos pacotes trafegados. Em ensaios comparativos, Rocha et al. [2018] demonstram que o OPC-UA impõe um consumo de banda e um tempo de resposta superiores quando confrontado com protocolos projetados para o tráfego leve de dados, fatores que limitam sua aplicabilidade em rotinas de sincronismo na infraestrutura de tecnologia da informação.

Por outro lado, a camada de orquestração do banco de dados exige alta velocidade para coordenar o consenso e monitorar a integridade dos servidores. Instabilidades temporárias na rede geográfica podem induzir o sistema a cometer erros de suspeita, gerando falsos positivos onde um nó saudável é erroneamente classificado como inativo devido ao atraso das mensagens. Esse fenômeno é analisado por Hayashibara et al. [2004] no desenvolvimento do detector de falhas Phi Accrual, o qual mitiga o impacto de flutuações de rede ao substituir a lógica binária de diagnóstico por uma escala contínua de nível de suspeita baseada no histórico de chegada das mensagens de controle. De maneira complementar, Das, Gupta e Motivala [2002] demonstram no protocolo SWIM que a escalabilidade e a eficiência no tráfego dessas mensagens de checagem são vitais para evitar o congestionamento dos canais de comunicação, reduzindo falhas induzidas pela própria sobrecarga do monitoramento.

Para otimizar o envio dessas validações periódicas de estado, conhecidas como heartbeats, os sistemas de alta disponibilidade utilizam frameworks de comunicação de alto desempenho como o gRPC, um framework do protocolo *Remote Procedure Call* (`RPC`). De acordo com Indrasiri e Kuruppu [2020], o `gRPC` otimiza a latência ao empregar os chamados *Protocol Buffers* para a serialização binária compacta dos dados, transmitindo as mensagens por meio da multiplexação nativa do protocolo *Hypertext Transfer Protocol 2* (`HTTP/2`). Essa eficiência técnica justifica o motivo pelo qual ferramentas de coordenação distribuída, a exemplo do etcd integrado ao Patroni, adotam o `gRPC` como protocolo

principal para a troca de mensagens de eleição de líder, mantendo o quórum do algoritmo Raft estável mesmo diante de pequenas variações na latência da rede.

3.6 Considerações do Capítulo

Como apresentado ao longo deste capítulo, a operação contínua é uma prioridade em ambientes críticos como os sistemas SCADA. Para garantir essa disponibilidade, apenas duplicar o hardware em diferentes sites não é suficiente, pois falhas de software, instabilidades de rede e o problema de particionamento de dados podem comprometer a integridade do sistema.

Diante dos desafios expostos, torna-se essencial utilizar um banco de dados relacional robusto, como o PostgreSQL, integrado a ferramentas de orquestração com capacidade de reconfiguração automática. Essas ferramentas devem ser fundamentadas em algoritmos de consenso tolerantes a falhas, como o Raft, e utilizar protocolos de comunicação de alta performance, como o `gRPC`, para tomar decisões de forma segura e distribuída. Com base nessa fundamentação teórica, o próximo capítulo detalhará a arquitetura proposta e os métodos que serão utilizados na prática para detectar falhas e realizar o *failover* contínuo no cenário industrial abordado neste trabalho.

4 Metodologia e Arquitetura Proposta

Este capítulo detalha a arquitetura do sistema desenvolvido, denominado *raft-ha-manager*, e os métodos empregados para garantir a alta disponibilidade do banco de dados PostgreSQL em um ambiente `SCADA`. Serão apresentadas as características do ambiente, os mecanismos de detecção de falhas, a dinâmica do algoritmo de consenso e a configuração do ambiente de simulação utilizado para validação.

4.1 Visão Geral da Solução

A solução proposta baseia-se na criação de um agente orquestrador customizado, o *raft-ha-manager*, desenvolvido na linguagem Java. Este agente é responsável por interligar o ciclo de vida do banco de dados PostgreSQL com uma implementação própria do protocolo de consenso Raft, utilizando chamadas de procedimento remoto via `gRPC` para comunicação entre os nós.

A topologia física real do ambiente `SCADA` alvo deste estudo é composta por 4 servidores, distribuídos geograficamente em dois locais físicos distintos, denominados Site A e Site B, com dois servidores em cada local. No entanto, para fins de validação e desenvolvimento neste trabalho, o sistema foi testado e homologado em um ambiente de simulação contendo três nós, sendo dois nós de dados que executam o serviço do *PostgreSQL* e um nó árbitro, cuja função é apenas prover quórum para as eleições do *cluster*, economizando assim recursos computacionais.

4.2 Caracterização e Detecção de Falhas

A garantia de alta disponibilidade requer que o sistema seja capaz de identificar prontamente quando um componente deixa de operar conforme o esperado.

4.2.1 Tipos de Falhas

No contexto de sistemas distribuídos industriais, as principais falhas mapeadas incluem falhas de *hardware* (como a queima de componentes ou perda total de energia no servidor), falhas de *software* (*crashes* abruptos do serviço do banco de dados) e problemas de conectividade de rede, que podem variar desde alta latência até o particionamento completo da rede, fenômeno que pode originar cenários de *split-brain*.

A seguir serão apresentadas duas imagens que demonstram um *cluster* de servidores antes e durante um particionamento de rede:

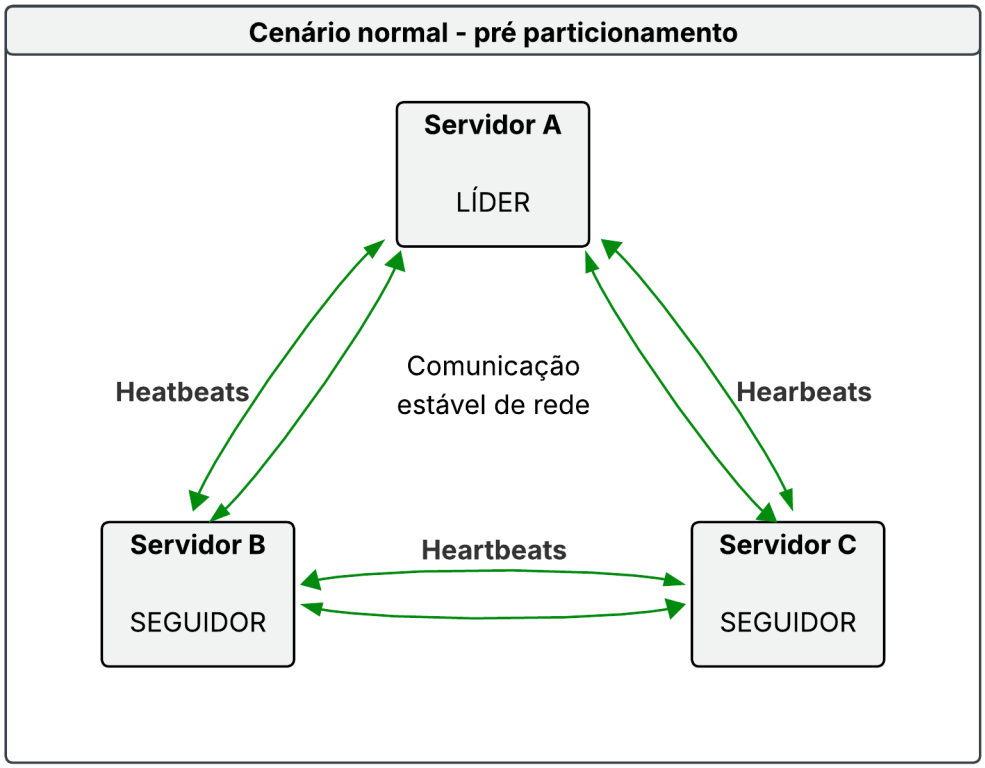


Figura 5 – Cluster de Servidores Antes do Particionamento

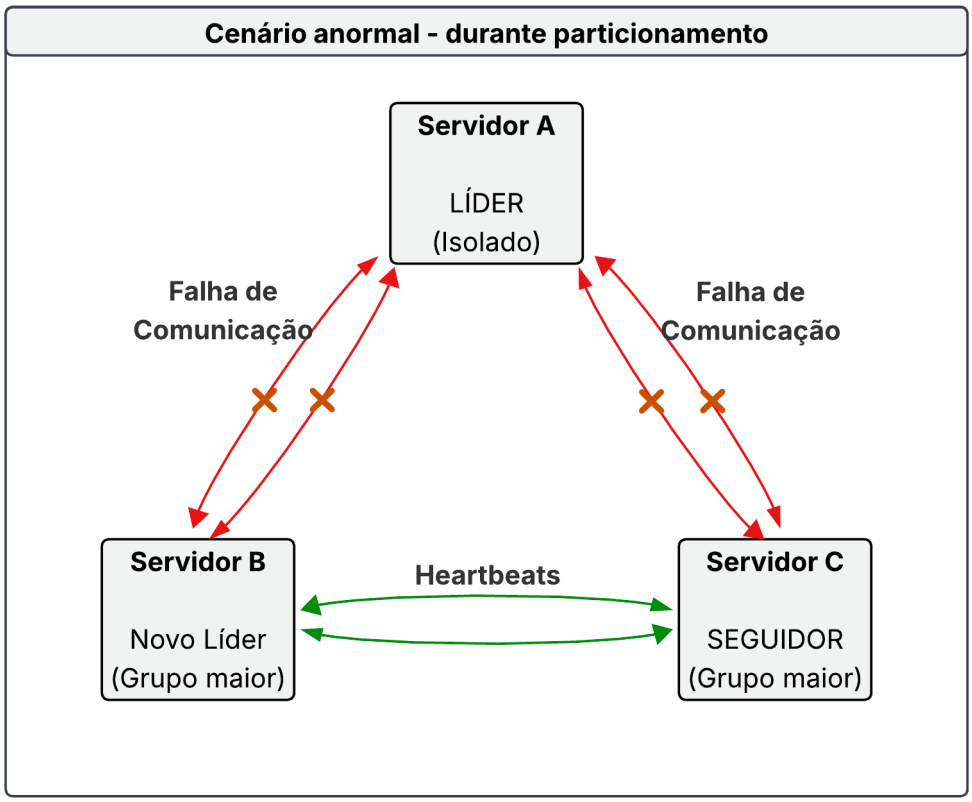


Figura 6 – Cluster de Servidores Durante um Particionamento

4.2.2 Mecanismos de Detecção

Para monitorar o estado do sistema, a arquitetura implementa duas camadas de detecção:

- **Monitoramento Local do Banco de Dados**: Através da classe *PostgresHealthChecker*, o agente orquestrador realiza testes de conexão via *Java Database Connectivity* (`JDBC`) no banco de dados local a cada três segundos. Se a conexão falhar por três vezes consecutivas, o banco é classificado como *UNHEALTHY* (não saudável).
- **Monitoramento do Cluster**: A integridade da rede e o status de cada nó são validados através dos *heartbeats* do algoritmo Raft, enviados pelo líder a cada cinco segundos via `gRPC`. A ausência desses sinais indica uma possível falha de rede ou do nó líder, dando início ao processo de eleição.

4.3 Caracterização do Ambiente SCADA

O ambiente `SCADA` que motivou o desenvolvimento desta solução possui restrições e características de alta criticidade. A infraestrutura física da empresa é dividida em dois locais distintos, denominados Site A e Site B. Cada um destes locais abriga dois servidores físicos, totalizando quatro nós na topologia real.

A necessidade de manter os dados operacionais absolutamente consistentes é inegociável, pois qualquer perda ou divergência nos dados pode impactar o controle do maquinário industrial. Embora a topologia física real consista em quatro nós, os testes, a implementação do *raft-ha-manager* e as simulações descritas neste trabalho foram realizados em uma arquitetura de três nós. Essa abordagem de simulação foi escolhida para validar a lógica fundamental do consenso e do *failover*, mantendo um quórum ímpar estrito, essencial para a tomada de decisão do protocolo Raft.

4.4 Arquitetura de Banco de Dados e Consenso

A integração entre o banco de dados PostgreSQL e o algoritmo de consenso é a peça central da solução.

A decisão de desenvolver a máquina de estados do Raft de forma manual, em vez de utilizar uma biblioteca pronta como o *Apache Ratis* ou o *etcd*, foi baseada em três pilares: o aprofundamento acadêmico exigido na pesquisa, a necessidade de integração refinada com o ciclo de vida do PostgreSQL e um maior controle sobre o comportamento de tolerância a falhas. Bibliotecas genéricas dificultariam a implementação de regras de

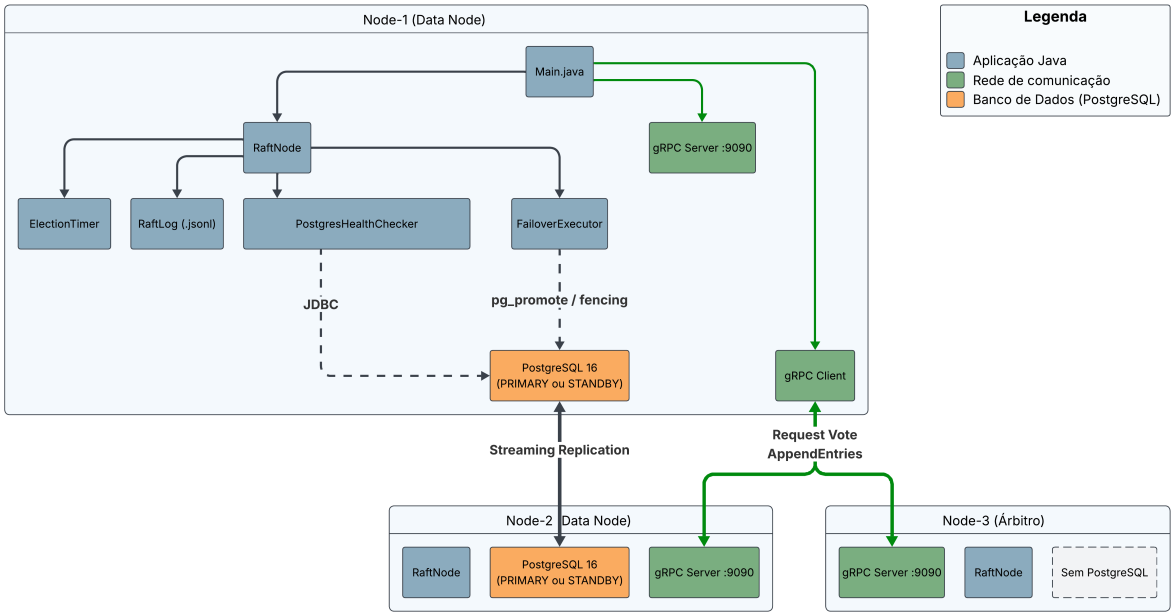


Figura 7 – Diagrama da Arquitetura do Sistema

negócio específicas, como bloquear a candidatura de um nó cujo banco de dados está fora do ar.

A comunicação entre os nós do cluster foi desenvolvida sobre o protocolo `gRPC`, utilizando serialização binária com *Protocol Buffers* rodando sobre HTTP/2. Essa escolha arquitetural provê uma latência de rede significativamente menor do que as tradicionais APIs baseadas em *Representational State Transfer* (`REST`) com *JavaScript Object Notation* (`JSON`), característica importante para o tráfego contínuo dos *heartbeats*. O armazenamento do *log* persistente do Raft, no entanto, foi estruturado em formato *JSON Lines* (`.jsonl`), pois permite a adição de novas entradas (*append-only*) de forma rápida, ao mesmo tempo em que preserva a legibilidade para fins de auditoria e depuração.

Apesar da praticidade e velocidade da adição contínua de registros, o acúmulo indiscriminado de logs ao longo do tempo levaria ao esgotamento do espaço em disco. Para contornar esse problema crítico de infraestrutura e garantir a viabilidade da solução a longo prazo, sistemas baseados no algoritmo Raft implementam rotinas de compactação de log ou *Snapshotting*. Neste processo, o estado atual do sistema é consolidado em um arquivo de captura pontual, também conhecido como *snapshot*, e todos os registros antigos do arquivo `.jsonl` que já foram replicados com segurança são descartados. Isso libera o espaço físico do disco dos servidores sem comprometer o histórico ou a consistência da máquina de estados do cluster.

4.5 Dinâmica de Failover

O processo de recuperação de falhas foi projetado para atuar rapidamente e, acima de tudo, tentar garantir a consistência dos dados, mitigando o problema de *split-brain*.

O link para acessar o fluxo de failover está disponibilizado em seguida: link do lucidchart

Quando o monitoramento local detecta que o PostgreSQL do nó líder falhou, ocorre um *Step-Down* voluntário. O nó líder renuncia imediatamente à sua posição, tornando-se um seguidor (*FOLLOWER*) e cessando o envio de *heartbeats*. Esse mecanismo previne que um líder sem banco de dados operacional trave o sistema. Com a interrupção dos *heartbeats*, os *election timeouts* ou temporizadores de eleição, que foram definidos com temporizadores entre quinze e trinta segundos, dos nós restantes expiram, iniciando uma nova eleição. O nó eleito então executa o utilitário `pg_promote`, elevando seu banco de dados local da condição de réplica (*STANDBY*) para primário (*PRIMARY*).

4.5.1 Prevenção de Split-Brain

Para cenários de particionamento de rede, onde um líder antigo isolado não tem conhecimento de que um novo líder foi eleito, essa situação foi descrita por Kleppmann [2017] como *delusional leader*, o sistema utiliza uma combinação de *Fencing* e *Self-Healing*:

- ***Fencing***: Se a máquina de estado do Raft de um nó estiver na condição de *FOLLOWER*, mas seu PostgreSQL local relatar estar como *PRIMARY*, o agente orquestrador intervém de forma forçada interrompendo o serviço do banco de dados local. Esse isolamento abrupto impede fisicamente que sistemas ou aplicações clientes interajam com um servidor diferente do primário, garantindo a integridade dos dados e seguindo o princípio *Fail-Safe* [KLEPPMANN, 2017];
- ***Self-Healing***: Imediatamente após a parada forçada, um processo de recuperação automática é acionado. Este script descarta o diretório de dados corrompido ou desatualizado e realiza um clone completo do banco de dados líder legítimo do cluster, reiniciando o serviço como um nó *STANDBY* seguro [TOEROE; TAM, 2012].

4.5.2 Redirecionamento de Clientes

A promoção interna de um novo nó ao papel de primário não é suficiente se as aplicações que interagem com os servidores não souberem para qual servidor enviar seus pedidos de gravação, busca, atualização e remoção de dados. Para solucionar a descoberta de serviço de forma prática, sem necessidade de reconfiguração manual nos sistemas, a

arquitetura prevê a utilização de um serviço que faz com que as aplicações sigam automaticamente o servidor primário. Quando ocorre um *failover* e um novo líder é estabelecido, mecanismos de rede assumem a responsabilidade de migrar o endereço automaticamente para a nova máquina primária. Com isso, as aplicações SCADA continuam disparando as requisições sempre para o mesmo endereço correto, enquanto o roteamento interno da rede cuida de direcionar o tráfego para o nó ativo naquele instante, diminuindo o tempo total de indisponibilidade dos serviços.

4.6 Ambiente de Simulação

O ecossistema utilizado para testar a aplicação consistiu em uma conjunto de três Máquinas Virtuais (`VM`) orquestradas no *software* VirtualBox. A automação da criação do ambiente foi realizada por meio do script `vm-create.ps1`, responsável por configurar as máquinas dentro de uma rede *Host-Only*, isolando o ambiente de interferências externas.

O processo de implantação e configuração do PostgreSQL na versão 16, e do *raft-ha-manager*, executado em uma *Java Virtual Machine* (`JVM`) 21, foi automatizado através de *shell scripts* executados de forma sequencial:

1. `01-base.sh`: realiza a configuração inicial do sistema operacional em todos os nós, definindo o endereço IP estático, ajustando as regras de *firewall*, editando o arquivo `/etc/hosts` para a resolução de nomes locais, além de instalar o Java e criar o usuário de serviço;

2. `02-postgres-primary.sh`: configura o PostgreSQL como primário no primeiro nó, ajustando os parâmetros de rede no arquivo `postgresql.conf` e as regras de acesso no arquivo `pg_hba.conf`, finalizando com a criação dos usuários de replicação;

3. `03-postgres-standby.sh`: atua nos demais nós de dados para configurá-los como réplicas, realizando a clonagem inicial através da ferramenta `pg_basebackup` e estabelecendo o canal de *streaming replication*;

4. `04-deploy-raft.sh`: implanta o executável Java da aplicação, distribui os arquivos de configuração no formato `YAML` e registra o *raft-ha-manager* como um serviço do *systemd*, garantindo a sua execução contínua em *background*.

Essa padronização garantiu que os testes de *failover*, *fencing* e *self-healing* fossem validados em um ambiente controlado e replicável, espelhando os desafios operacionais do ambiente `SCADA` industrial.

4.6.1 Limitações da Simulação

A literatura acadêmica clássica do algoritmo Raft, concebida por Ongaro e Ousterhout [2014], determina que os *heartbeats* devem ser enviados em intervalos de 10 a 50 milissegundos com *timeouts* de eleição oscilando entre 150 e 300 milissegundos. Tais valores garantem um tempo de inatividade da rede quase imperceptível, requisito importante para um ambiente industrial.

Contudo, exclusivamente para a prova de conceito e validação funcional neste trabalho acadêmico, os valores foram configurados de forma conservadora: *heartbeats* de 5 segundos e temporizadores de eleição entre 15 e 30 segundos. Estes ajustes nos tempos foram necessários para compensar a limitação introduzida pelo próprio ambiente de simulação. Como os testes foram realizados em `VM`s compartilhando recursos do mesmo processador físico, somando isso a execução do orquestrador na `JVM`, o sistema tornou-se suscetível a pequenas pausas de escalonamento de processos do hospedeiro e a rotinas de coleta de lixo, processo que ocorre automaticamente para limpar memória nas `JVM`s. Utilizar os valores de milissegundos no ambiente virtualizado geraria latências falsas e expirações prematuras, provocando eleições de líder desnecessárias e instabilidade no *cluster*. Dessa forma, a adoção de valores maiores permitiu validar com maior facilidade e segurança a lógica de *failover* sem essas pequenas interferências da virtualização.

5 Testes Realizados

Este capítulo tem como objetivo demonstrar, através de cenários de injeção de falhas, a eficácia da arquitetura proposta. Os testes práticos foram executados em um ambiente de simulação composto por três nós.

5.1 Análise do Cenário 1: Falha no Banco de Dados

O primeiro cenário de teste consistiu na simulação de uma falha completa do processo do PostgreSQL no nó líder. O objetivo foi aferir a capacidade do *raft-ha-manager* de detectar a queda do serviço, revogar a liderança localmente, convocar uma nova eleição no *cluster* e promover um nó *STANDBY* a *PRIMARY*.

O teste foi executado em três rodadas consecutivas para garantir a consistência dos resultados. A Tabela 3 apresenta os tempos observados para cada etapa do processo de *failover*.

Tabela 3 – Métricas aferidas no cenário de Falha no Banco de Dados.

Rodada	Tempo de Detecção	Tempo de Eleição	MTTR	Recuperação
1	6 s	16 s	24 s	$\sim$ 8 s
2	6 s	21 s	29 s	$\sim$ 8 s
3	6 s	16 s	24 s	$\sim$ 8 s
Média	**6.0 s**	**17.6 s**	**25.6 s**	$\sim$ **8.0 s**

Fonte: Autor.

Os resultados numéricos demonstram a conformidade com o comportamento matemático esperado do algoritmo. A detecção da falha em 6 segundos é explicada pelo ciclo de *polling* do *PostgresHealthChecker*, que opera com um intervalo de 3 segundos e exige duas falhas consecutivas antes de agir.

A variabilidade do tempo de eleição, oscilando entre 16 e 21 segundos, ocorreu dentro da janela de randomização estipulada no código de 15 a 30 segundos, um mecanismo desenvolvido originalmente por Ongaro e Ousterhout [2014] para mitigar o risco de *split votes*.

O tempo médio de recuperação, MTTR, de 25,6 segundos comprova que o sistema funcionou como projetado. Em um cenário real de produção, a janela de eleição e o tempo de *polling* seriam ajustados para a casa dos milissegundos, reduzindo o MTTR e mantendo a garantia e integridade arquitetural provada por este teste.

5.2 Análise do Cenário 2: Particionamento de Rede

O segundo cenário validou a resiliência do sistema frente ao isolamento total de rede do nó líder, a falha que culmina no problemático estado de *Split-Brain*. Foram realizadas três rodadas utilizando regras de bloqueio de tráfego TCP na porta de comunicação do gRPC.

A Tabela 4 expõe os resultados observados durante o período de isolamento e após o restabelecimento da rede.

Tabela 4 – Métricas aferidas no cenário de Particionamento de Rede.

Rodada	**Tempo de Eleição**	**MTTR**	**Self-Healing**
1	$\sim$ 20 s	$\sim$ 22 s	8 s
2	$\sim$ 20 s	$\sim$ 22 s	8 s
3	$\sim$ 15 s	$\sim$ 17 s	8 s
Média	$\sim$ **18.3 s**	$\sim$ **20.3 s**	**8.0 s**

Fonte: Autor.

Nesse teste, a distinção entre o tempo de eleição, ocorrido na partição saudável da rede, e o tempo de reconstrução local, *Fencing* e *Self-Healing* ocorridos na reconexão do nó isolado, foi separada. Essa escolha comprova de forma numérica que o sistema não apenas percebe a anomalia elegendo um novo líder em média após 18,3 segundos, mas consegue estancar a divergência de dados e forçar o retorno a um estado íntegro em tempo hábil.

Quando a conectividade do antigo líder foi restaurada, a rotina de isolamento interrompeu o banco de dados desatualizado e desencadeou a clonagem automatizada do líder legítimo, via *pg_basebackup*. Esse processo destrutivo e corretivo durou em média 8 segundos, demonstrando eficiência na mitigação do *Split-Brain*, prescindindo de intervenção humana.

5.3 Limitações da Arquitetura

Apesar do êxito na orquestração da falha e recuperação, a análise dos *logs* e do protocolo revelou limitações na arquitetura atual que devem ser discutidas.

5.3.1 Modelo Crash Fault Tolerance

Do ponto de vista avaliativo, é importante reconhecer que o algoritmo Raft e a solução construída atuam sob a premissa de um modelo *Crash Fault Tolerant*(`CFT`) [ONGARO; OUSTERHOUT, 2014]. O algoritmo foi concebido para suportar a interrupção de processos, atrasos de mensagens e o isolamento de rede, pressupondo, contudo, que os nós sempre enviem informações verdadeiras. Caso a infraestrutura sofra uma falha

bizantina, isto é, caso um nó minta sobre seus valores, altere votos ou encaminhe pacotes corrompidos, intencionalmente ou não, o sistema não garante a consistência do consenso. A solução atual não implementa tolerância a falhas bizantinas, detalhada em protocolos avançados como o de Castro e Liskov [1999], o que a torna vulnerável a certos ataques maliciosos no âmbito industrial.

5.3.2 Problema de Reconexão

A observação prática durante a primeira rodada do particionamento de rede diagnosticou um comportamento esperado no algoritmo base. Ao ser isolado, o antigo nó líder elevou continuamente o seu mandato, ou *term*, no escuro. Na reconexão, esse mandato artificialmente alto forçou o líder legítimo do *cluster* a abdicar temporariamente de sua posição, ocasionando uma oscilação de instabilidade transiente.

A causa desta oscilação é a ausência da fase de pré-votação, o mecanismo conhecido como *Pre-Vote*. Sem ele, não há um bloqueio do sistema que impeça o incremento de mandato durante o isolamento. A possibilidade de observar essa limitação do algoritmo mostra que a implementação do algoritmo foi bem sucedida, tendo o *raft-ha-manager* simulado essa mesma limitação. A implementação do mecanismo de *Pre-Vote* como uma melhoria na aplicação seria um próximo passo a ser melhor explorado em trabalhos futuros.

5.4 Considerações Finais do Capítulo

Esta análise mostra o valor prático da arquitetura construída em comparação com o esforço de uma intervenção manual. A delegação da alta disponibilidade para o protocolo de consenso Raft reduziu o tempo de recuperação total de sistemas, que normalmente demoraria pelo menos uma hora, para alguns segundos. A orquestração protege a base de dados com integridade e mostra a implementação da tese central defendida por Ongaro e Ousterhout [2014] frente aos sistemas distribuídos.

6 Conclusões e Trabalhos Futuros

Referências

2NDQUADRANT. *repmgr: The Most Advanced Replication Manager for PostgreSQL*. 2026. Disponível em: <https://repmgr.org>. Citado na página 16.

AILIJIANG, A.; CHARAPKO, A.; DEMIRBAS, M. Dissecting the performance of strongly-consistent replication protocols. In: *Proceedings of the 2019 International Conference on Management of Data*. New York, NY, USA: Association for Computing Machinery, 2019. (SIGMOD '19), p. 1696–1710. ISBN 9781450356435. Disponível em: ⟨https://doi.org/10.1145/3299869.3319893⟩. Citado na página 14.

ALANAZI, M.; MAHMOOD, A.; CHOWDHURY, M. J. M. Scada vulnerabilities and attacks: A review of the state-of-the-art and open issues. *Computers and Security*, v. 125, p. 103028, 2023. ISSN 0167-4048. Disponível em: ⟨https://www.sciencedirect.com/science/article/pii/S0167404822004205⟩. Citado 2 vezes nas páginas 7 e 8.

BHARDWAJ, R.; DATTA, D. Consensus algorithm. In: _____. Springer International Publishing, 2020. ISBN 978-3-030-38677-1. Disponível em: ⟨https://doi.org/10.1007/978-3-030-38677-1_5⟩. Citado na página 10.

CASTRO, M.; LISKOV, B. Practical byzantine fault tolerance. *Proceedings of the Symposium on Operating System Design and Implementation*, 1999. Citado na página 28.

DAS, A.; GUPTA, I.; MOTIVALA, A. Swim: Scalable weakly-consistent infection-style process group membership protocol. In: *Proceedings of the 2002 International Conference on Dependable Systems and Networks*. [S.l.]: IEEE Computer Society, 2002. (DSN '02), p. 303–312. ISBN 0769515975. Citado na página 17.

DATA, C. *pg_auto_failover: PostgreSQL Automatic Failover*. 2026. Disponível em: ⟨https://github.com/citusdata/pg_auto_failover⟩. Citado na página 16.

DRAHOš, P. et al. Trends in industrial communication and opc ua. In: *2018 Cybernetics & Informatics (K&I)*. [S.l.: s.n.], 2018. p. 1–5. Citado na página 17.

GRAY, J.; SIEWIOREK, D. P. High-availability computer systems. *Computer*, 1991. Citado na página 9.

GROUP, T. P. G. D. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. 2026. Disponível em: ⟨https://www.postgresql.org⟩. Citado na página 14.

HALL, M. et al. The weka data mining software: An update. *SIFKDD Explorations*, v. 11, 2009. Citado na página 5.

HAYASHIBARA, N. et al. The /spl phi/ accrual failure detector. In: *Proceedings of the 23rd IEEE International Symposium on Reliable Distributed Systems, 2004.* [S.l.: s.n.], 2004. p. 66–78. Citado na página 17.

INDRASIRI, K.; KURUPPU, D. *gRPC: Up and Running: Building Cloud Native Applications with Go and Java for Docker and Kubernetes*. [S.l.]: "O'Reilly Media, Inc.", 2020. Citado na página 17.

JONES, E. et al. *SciPy: Open source scientific tools for Python*. 2001. Disponível em: ⟨http://www.scipy.org⟩. Citado na página 5.

KLEPPMANN, M. *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. [S.l.]: "O'Reilly Media, Inc.", 2017. Citado 2 vezes nas páginas 11 e 23.

LAMPORT, L. Paxos made simple. *ACM SIGACT News (Distributed Computing Column) 32, 4 (Whole Number 121, December 2001)*, p. 51–58, 2001. Citado 3 vezes nas páginas 10, 11 e 14.

LAMPORT, L.; SHOSTAK, R.; PEASE, M. The byzantine generals problem. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 1982. Citado na página 11.

ONGARO, D.; OUSTERHOUT, J. *In Search of an Understandable Consensus Algorithm (Extended Version)*. [S.l.], 2014. Citado 6 vezes nas páginas 13, 14, 25, 26, 27 e 28.

RAJESH, S. C.; KUMAR, D. R. High availability strategies in distributed systems: A practical guide. *ResearchGate, January*, 2025. Citado 2 vezes nas páginas 8 e 9.

ROCHA, M. et al. Performance comparison between opc ua and mqtt for data exchange. In: *Performance Comparison Between OPC UA and MQTT for Data Exchange*. [S.l.: s.n.], 2018. p. 175–179. Citado na página 17.

SE, Z. *Patroni: A Template for PostgreSQL High Availability with ZooKeeper, etcd or Consul*. 2026. Disponível em: ⟨https://patroni.readthedocs.io⟩. Citado na página 16.

SOARES, J. D. R. *Modelagem Matemática na Previsão de Colheita de Bananeira: Regressão Linear Múltipla x Redes Neurais Artificiais*. Tese (Doutorado) — Universidade Federal de Lavras, 2010. Citado na página 10.

TOEROE, M.; TAM, F. *Service availability: principles and practice*. [S.l.]: John Wiley & Sons, 2012. Citado 2 vezes nas páginas 9 e 23.

UNIOESTE. *Universidade Estadual do Oeste do Paraná*. 2017. Disponível em: ⟨http://www5.unioeste.br/portalunioeste/⟩. Citado 2 vezes nas páginas 2 e 3.

YADAV, G.; PAUL, K. *Architecture and security of SCADA systems: A review*. 2021. Citado 2 vezes nas páginas 7 e 8.